

Proxy Server - Reverse Proxy

1. *Proxy Server (Forward Proxy)*: đại diện cho Người dùng (Client).

- Vị trí: đứng trước các máy tính trong mạng nội bộ.
- Cách hoạt động: khi Client muốn truy cập vào <https://fit.iuh.edu.vn>, máy tính của Client gửi yêu cầu đến Proxy. Proxy sẽ thay mặt Client đi hỏi, lấy dữ liệu về rồi đưa lại cho Client.
- Mục đích:
 - Ẩn danh: Server (<https://fit.iuh.edu.vn>) chỉ thấy IP của Proxy, không thấy IP thật của Client.
 - Vượt rào/Kiểm soát: công ty có thể dùng Proxy để chặn nhân viên vào Youtube, hoặc Client dùng Proxy để vào các trang web bị chặn địa lý.
 - Caching: lưu trữ những trang web hay dùng để tải nhanh hơn cho người dùng trong mạng.

2. *Reverse Proxy (Server đảo ngược)*: đại diện cho Máy chủ (Server).

- Vị trí: đứng trước một hoặc một cụm máy chủ Backend.
- Cách hoạt động: người dùng từ khắp nơi trên Internet gửi yêu cầu đến địa chỉ website. Họ thực chất đang kết nối tới Reverse Proxy. Sau đó, Reverse Proxy mới điều hướng yêu cầu đó vào các server bên trong.
- Mục đích:
 - Load Balancing (cân bằng tải): chia đều công việc cho nhiều máy chủ.
 - Bảo mật: giấu kín IP thật của các máy chủ Backend. Hacker chỉ có thể tấn công vào Reverse Proxy, không thể chạm trực tiếp vào "core" hệ thống.
 - SSL Termination: giải mã HTTPS tại Reverse Proxy để giảm tải tính toán cho các Server chạy App phía sau.

Thuật toán điều phối (Algorithms)

Đây là "Core logic" quyết định request sẽ đi đâu:

1. Static Algorithms: theo quy tắc lập trình sẵn, không quan tâm đến tình trạng sức khỏe thực tế của server (như CPU, RAM).

- Round Robin: chia đều theo vòng tròn ($A \rightarrow B \rightarrow C$).
 - Cách chạy:
 - Request 1 vào Server A
 - Request 2 vào Server B
 - Request 3 vào Server C
 - Rồi quay lại A.
 - Khi nào dùng: khi các server có cấu hình phần cứng y hệt nhau và các tác vụ xử lý có độ nặng tương đương.
- Weighted Round Robin: chia theo trọng số (Máy mạnh nhận nhiều hơn).
 - Cách chạy: gán cho mỗi server một "trọng số" (weight).

Ví dụ:

- Server A (mạnh) weight=3
- Server B (yếu) weight=1
 - Nginx sẽ đẩy 3 request vào A rồi mới đẩy 1 request vào B
- Khi nào dùng: khi có nhiều server không giống nhau về cấu hình (server cũ, server mới, server thuê cấu hình cao, server thấp).
- IP Hash: cố định một khách hàng vào một server nhất định (giữ Session).
 - Cách chạy: nginx lấy địa chỉ IP của khách hàng thực hiện một phép toán (hash) để ra một con số, con số này tương ứng với một server cố định.
 - Khi nào dùng: khi ứng dụng cần Sticky Session.

Ví dụ: người dùng đang add item vào Cart (Giỏ hàng) trên Server A, nếu request sau nhảy sang Server B mà server này chưa kịp đồng bộ session, giỏ hàng sẽ bị trống.

2. *Dynamic Algorithms*: nginx sẽ "quan sát" hiệu năng thực tế để điều phối

- Least Connections: đẩy vào máy đang có ít người dùng nhất
 - Cách chạy: nginx đếm xem mỗi server đang có bao nhiêu kết nối. Request mới sẽ được đẩy vào server đang "rảnh" nhất.
 - Khi nào dùng: khi các request có thời gian xử lý khác nhau.
Ví dụ: Một request chỉ để xem ảnh (nhANH), nhưng một request khác lại để xuất file Excel (lâu). Round Robin sẽ bị quá tải nếu vô tình dồn nhiều request nặng vào một máy, nhưng Least Connections thì không.
- Least Response Time: đẩy vào máy đang phản hồi nhanh nhất.
 - Cách chạy: nginx tính toán thời gian từ lúc gửi request đến lúc server trả về dữ liệu (latency) và cộng với số kết nối hiện có. Máy nào "phản hồi" nhanh nhất sẽ được ưu tiên.
 - Khi nào dùng: khi muốn tối ưu tối đa trải nghiệm người dùng (UX)

Nginx

https://nginx.org/en/docs/beginners_guide.html

1. Cơ chế Load Balancer với Nginx là gì?

- Load Balancer như một "người điều phối giao thông" đứng trước một nhóm máy chủ (Server Pool). Khi người dùng gửi yêu cầu (request) đến website, thay vì yêu cầu đó đi thẳng vào server xử lý, nó sẽ đi qua Nginx.
- Nginx sẽ xem xét: "Server nào đang rảnh? Server nào có hiệu năng tốt nhất?" và chuyển tiếp request đó đến đúng nơi.

2. Tại sao lại dùng Nginx?

- Tính sẵn sàng cao (High Availability): nếu một server bị chết, Nginx sẽ tự động nhận diện và ngừng gửi request đến đó, giúp hệ thống không bị gián đoạn.
- Khả năng mở rộng (Scalability): có thể dễ dàng thêm nhiều server nữa vào cụm khi lượng traffic tăng đột biến mà không cần thay đổi code ứng dụng.
- Offloading: nginx có thể đảm nhận việc nén dữ liệu hoặc giải mã SSL, giúp giảm tải tài nguyên cho các server phía sau.

3. Cấu hình chi tiết

Ví dụ: giả sử có **3 máy chủ (Server)** đang chạy ứng dụng web tại các địa chỉ IP: 192.168.1.10, 192.168.1.11, 192.168.1.12

Cấu hình Nginx đóng vai trò là "cổng vào" (Entry Point) tại địa chỉ 192.168.1.5.

Tạo file nginx.conf ở etc/nginx/nginx.conf hoặc etc/nginx/conf.d

```
http {  
    # 1. Định nghĩa nhóm các server xử lý (Upstream Group)  
    upstream my_web {  
        server 192.168.1.10:8080 weight=3;  
        server 192.168.1.11:8080;  
        server 192.168.1.12:8080 backup;  
    }  
  
    server {
```

```
listen 80;
server_name mywebsite.com;

location / {
    # 2. Điều hướng request đến nhóm upstream đã định nghĩa
    proxy_pass http://my_web;

    # 3. Giữ lại thông tin gốc của người dùng
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
}
```

Traefik

<https://doc.traefik.io/traefik/>

1. Traefik là gì?

Traefik là một **Edge Router** (Bộ định tuyến biên) hiện đại và là một Reverse Proxy/Load Balancer mã nguồn mở. Điểm khác biệt lớn nhất của nó so với Nginx là khả năng **tự động cấu hình (Auto-configuration)**.

2. Sự khác biệt so với Nginx

Trong kiến trúc Microservices, các server (container) thường xuyên được bật lên hoặc tắt đi, địa chỉ IP thay đổi liên tục.

- **Với Nginx:** mỗi lần thêm một server mới, phải vào sửa file nginx.conf thủ công rồi reload. Rất mất thời gian và dễ sai sót. (đối với cách add server tĩnh)
- **Với Traefik:** có cơ chế **Service Discovery**. Traefik sẽ "lắng nghe" Docker hoặc Kubernetes. Khi một container mới được chạy lên, Traefik tự động nhận diện và đưa nó vào danh sách Load Balancing ngay lập tức mà không cần phải sửa file cấu hình.

3. Các thành phần Core (lõi) của Traefik

- a. **Providers:** là nơi Traefik lấy thông tin về các server (ví dụ: Docker, Kubernetes, File). Traefik sẽ "hỏi" Provider: "Hiện tại có những dịch vụ nào đang chạy?".
- b. **Entrypoints:** là các "cửa ngõ" đón request (ví dụ: Cổng 80 cho HTTP, cổng 443 cho HTTPS).
- c. **Routers:** phân tích request (URL là gì? Header có gì?) để quyết định xem request này thuộc về dịch vụ nào.
- d. **Services:** là các Backend Server (các container). Traefik sẽ thực hiện Load Balancing tại đây.
- e. **Middlewares:** cho phép chỉnh sửa request trước khi gửi đến Service (ví dụ: Thêm tiền tố, kiểm tra quyền đăng nhập, nén dữ liệu...).

4. Ưu điểm nổi bật

- Tự động cấp chứng chỉ SSL (Let's Encrypt): Traefik tự động xin và gia hạn SSL cho website, không cần cài thêm Certbot như Nginx.
- Giao diện Dashboard: Traefik đi kèm một giao diện web trực quan để theo dõi luồng traffic và tình trạng các service theo thời gian thực.
- Hỗ trợ HTTP/3 và gRPC: sẵn sàng cho các công nghệ web mới nhất.

```
services:
  traefik:
    image: traefik:v3.3.4
    command:
      - "--api.insecure=true" # Mở Dashboard (cổng 8080)
      - "--providers.docker=true" # Cho phép Traefik tự quét Docker
      - "--entrypoints.web.address=:80" # Mở cổng 80 để nhận traffic
    ports:
      - "80:80" # Cổng HTTP chính
      - "8080:8080" # Cổng Dashboard (chỉ dùng khi dev)
    volumes:
      - "/var/run/docker.sock:/var/run/docker.sock:ro" # Cho phép Traefik
        "đọc" các container khác

  webapp:
    image: traefik/whoami # Một web app trả về thông tin server
    deploy:
      replicas: 2 # chạy 2 server cùng lúc để test Load Balancing
    labels:
      - "traefik.http.routers.my-app.rule=Host(`localhost`)"
      - "traefik.http.services.my-app.loadbalancer.server.port=80"
```